IST604 – Web Services & Distributed Computing

Chapter 2: Structural & Essay Questions — Full Model Answers

Part I: Structural / Short-Answer Questions

Q1. Define a web service and list four key characteristics that distinguish it from other software components. *(4 marks)*

Answer:

A **web service** is a standards-based, language-agnostic software entity that accepts specially formatted requests from other software entities on remote machines via vendor and transport-neutral communication protocols, and produces application-specific responses. It enables two or more applications — potentially written in different programming languages and running on different platforms — to communicate and exchange data over a network using standardized protocols such as HTTP.

Four key characteristics:

1. **Standards-based** — built on universally accepted, open specifications (e.g., W3C standards such as SOAP, WSDL, XML), ensuring broad compatibility across implementations.
2. **Language-agnostic** — a Java application and a .NET application can communicate through a web service without either side knowing or caring about the other's programming language.
3. **Vendor-neutral** — the web service model is not tied to any particular vendor's platform, product, or proprietary technology. Any vendor-compliant implementation can participate.
4. **Transport-neutral** — web services can operate over multiple transport protocols (HTTP, SMTP, FTP, TCP). The service logic is independent of the transport layer used to carry messages.

(Additional characteristics include: formatted requests, application-specific responses, and remote machine operation.)

Q2. Differentiate between a web service and an API with respect to: (a) network dependency, and (b) scope of applicability. *(4 marks)*

Answer:

(a) Network Dependency:

A **web service** always requires a network connection to function. It is designed to operate over a network — either the internet or an intranet — using network protocols such as HTTP. Without a network, a web service cannot be invoked.

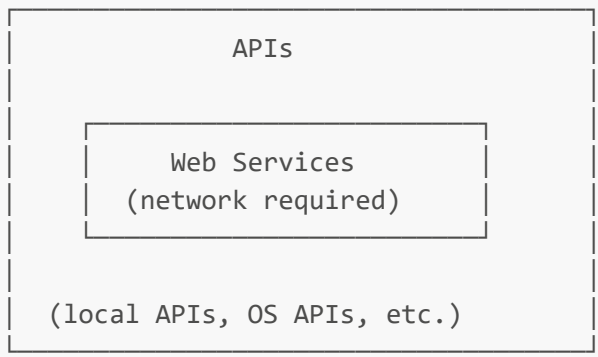
An **API** (Application Programming Interface) does not necessarily require a network. An API can be entirely local, such as the Windows API or a library bundled within an operating system. It defines a programming

interface that can be called within the same machine, process, or application without any network involvement.

(b) Scope of Applicability:

All web services are APIs because they expose a programmatic interface for interaction. However, not all APIs are web services. The term "API" is broader: it encompasses local OS APIs, hardware driver interfaces, library APIs, and remote/web-based interfaces alike. A web service is a specialised subset of APIs that specifically operates over a network using standardized web protocols.

Summary:



Q3. Identify and briefly explain the three core functionality features of web services. *(6 marks)*

Answer:

1. Interoperability

Web services act as a communication bridge between applications built on different programming languages and running on different platforms. For example, a Java application running on a Linux server can seamlessly communicate with a .NET application running on a Windows machine through a web service. The web service abstracts away the language and platform differences, translating requests and responses into a mutually understood format (typically XML or JSON over HTTP). This is the most fundamental capability of a web service.

2. Standardized Messaging

Data exchanged between a service provider and a service consumer is structured using standardized, universally understood formats. The two dominant formats are:

- **XML** (eXtensible Markup Language): a verbose but powerful, self-describing markup language used in SOAP-based services.
- **JSON** (JavaScript Object Notation): a lightweight, human-readable key-value format increasingly preferred in RESTful services.

These standards ensure that any compliant system can parse and understand the messages, regardless of the technology stack being used.

3. Loose Coupling

Loose coupling means the client and server are independent of each other's internal implementation. A change in the server's internal code — such as refactoring a database query or changing a business logic algorithm — does not require any change in the client, provided the service interface (the contract) remains the same. This independence is crucial for maintainability and scalability in distributed systems. It also allows services to be updated, replaced, or redeployed without disrupting consumers.

Q4. State three benefits of using web services in enterprise application integration. *(3 marks)*

Answer:

1. Loose Coupling As discussed above, the independence between client and server means that enterprise systems can evolve independently. Internal changes on one side do not break the other, reducing integration fragility and maintenance costs.

2. Ease of Integration In large organisations, different departments or systems often store data in isolated silos — each application manages its own data independently. Web services act as the "glue" that connects these silos, enabling data and functionality to be shared across applications and even across organisational boundaries without requiring applications to be rebuilt or merged.

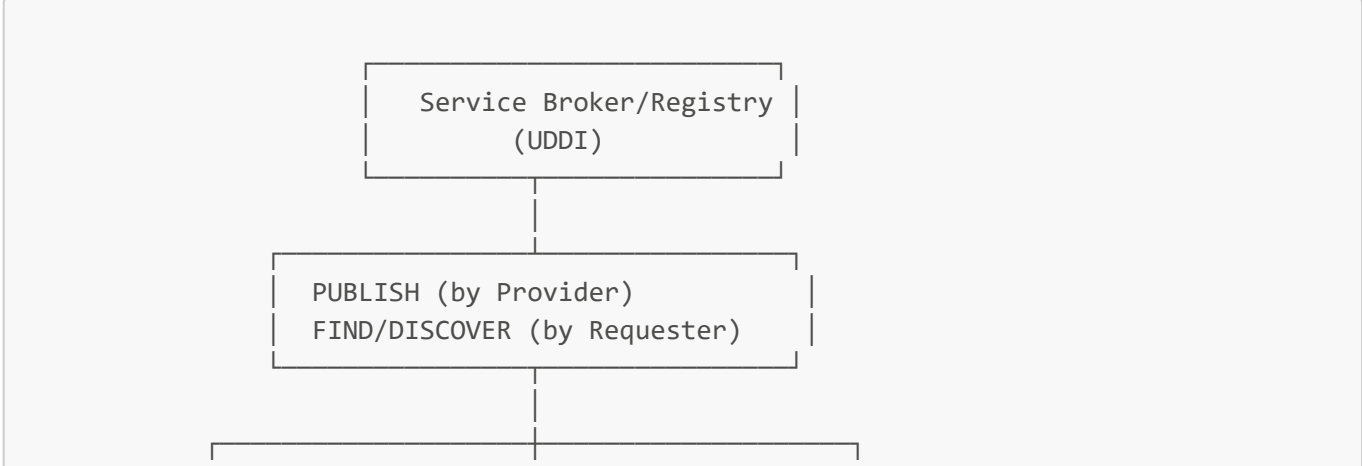
3. Service Reuse Web services extend the concept of code reuse to a network level. A specific business function (e.g., currency conversion, identity verification, tax calculation) needs to be implemented only once as a web service. Multiple consuming applications — potentially built by different teams in different languages — can then reuse that service over the network, eliminating redundant implementation and ensuring consistency.

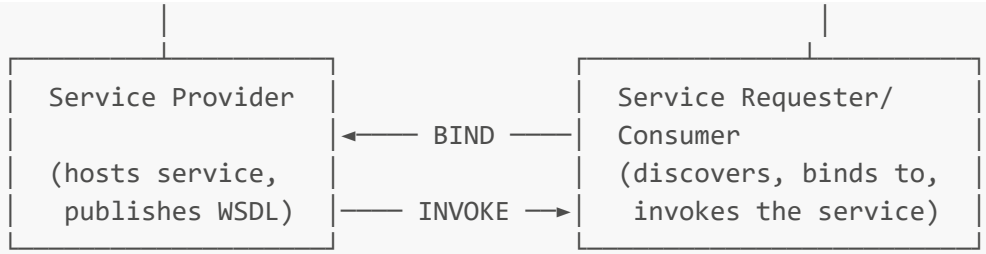
Q5. With the aid of a labeled diagram, explain the three-role SOA model for web services, describing the responsibility of each role. *(9 marks)*

Answer:

Service-Oriented Architecture (SOA) defines a three-role model for how web services are published, discovered, and consumed. The three roles are the **Service Provider**, the **Service Broker/Registry**, and the **Service Requester/Consumer**.

Diagram:





Role 1: Service Provider

The Service Provider is responsible for developing, deploying, and hosting the web service. It defines the service operations and their interfaces, and publishes a formal description of those interfaces (a WSDL document) to the Service Registry so that potential consumers can discover and understand the service. The Provider owns and manages the service endpoint.

Role 2: Service Broker / Registry

The Service Broker (commonly implemented using UDDI — Universal Description, Discovery, and Integration) acts as a directory or catalogue of available web services. Service Providers register their services (and their WSDL descriptions) in this registry. Service Requesters query the registry to discover available services. The Broker facilitates the match between providers and consumers by maintaining metadata about service types, descriptions, and locations.

Role 3: Service Requester / Consumer

The Service Requester is the client application that needs to use a web service. It queries the Service Broker to find a suitable service, retrieves the WSDL description of that service, and uses that description to bind to the Provider's endpoint. Once bound, it invokes the service operations by sending appropriate requests (typically SOAP messages) and receives the responses.

Summary of Interactions:

- Provider → Registry: **PUBLISH** (registers service + WSDL)
- Requester → Registry: **FIND** (discovers service, obtains WSDL)
- Requester → Provider: **BIND and INVOKE** (connects to endpoint and calls operations)

Q6. Compare the RPC-based and Messaging-based communication models for web services across coupling, synchronicity, and data transmission unit. *(6 marks)*

Answer:

1. Coupling:

- **RPC-Based**: Tightly coupled. The client must know the specific remote methods available and their signatures. The client is implemented with remote objects that mirror the service's method structure, meaning any change to the service's operations directly affects the client.
- **Messaging-Based**: Loosely coupled. The client and server interact via document exchange. The client only needs to know the message format (the document structure), not the specific implementation methods of the service. Services can evolve internally without breaking document-based clients.

2. Synchronicity:

- **RPC-Based**: Strictly synchronous. When the client sends a request, it must wait (blocking its own execution) until the server processes the request and sends back a response before the client can proceed. This is modelled after a traditional function call. Examples: CORBA, RMI.
- **Messaging-Based**: Can be synchronous or asynchronous. In the asynchronous variant, the client sends a message (document) into a message queue and continues its own execution without waiting. The server processes the message in its own time and may or may not send a response. This is particularly suited to long-running or batch operations.

3. Data Transmission Unit:

- **RPC-Based**: Transmits a set of discrete **parameters** (analogous to function arguments). The client sends individual parameter values; the server executes the corresponding method with those parameters and returns the return value.
- **Messaging-Based**: Transmits an entire **document** (a self-contained XML or structured message). Rather than passing parameters to a function, the client packages the relevant data into a document and submits it in full. The server parses and processes the document holistically.

Comparison Table:

Dimension	RPC-Based	Messaging-Based
Coupling	Tight	Loose
Synchronicity	Always synchronous	Synchronous or asynchronous
Data unit	Parameters/method calls	Entire documents
Examples	CORBA, RMI	JMS, message queues

Q7. What is the role of UDDI in a Service-Oriented Architecture? How does it interact with WSDL and SOAP? *(6 marks)*

Answer:

Role of UDDI:

UDDI (Universal Description, Discovery, and Integration) is an XML-based framework that acts as the **service registry** in an SOA ecosystem. Its primary role is to provide a centralised directory where:

- **Service Providers** can publish and categorise their web services, making them discoverable.
- **Service Requesters** can search and find services that match their needs.

UDDI maintains metadata about registered services, including service types, textual descriptions, and the locations (endpoints) of the services. Web service brokers use UDDI as the standard mechanism for registering provider information.

Interaction with WSDL:

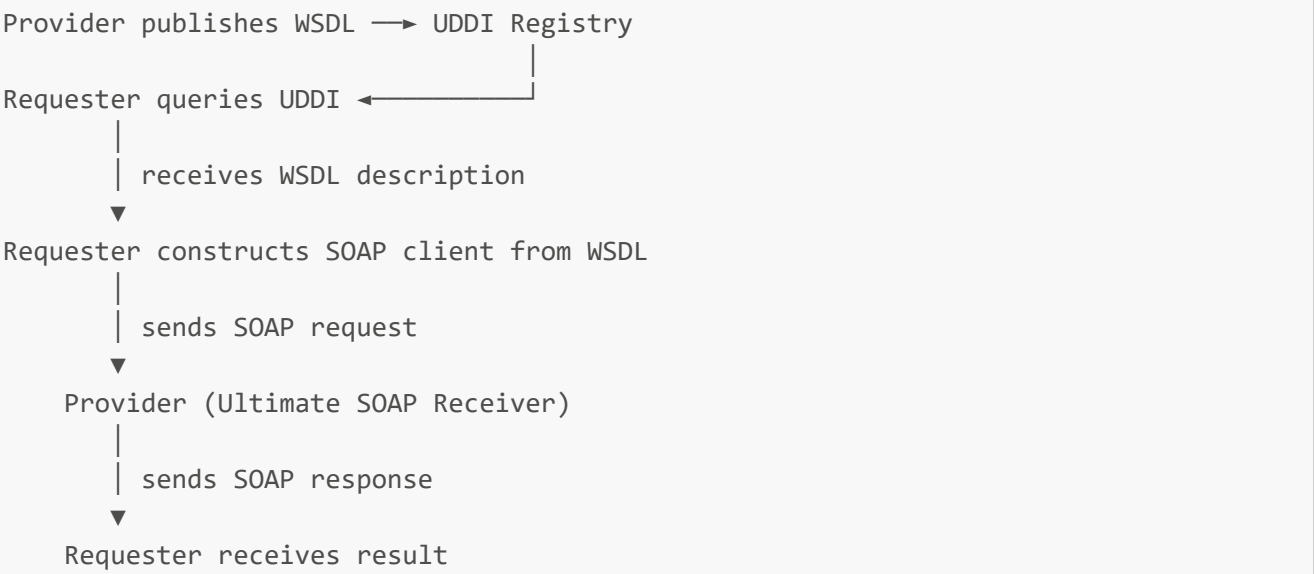
When a service provider registers a service in UDDI, it publishes a reference to the service's WSDL document. The UDDI entry contains pointers to the WSDL. When a service requester queries the UDDI registry for a

particular service, the registry returns the corresponding WSDL description of the service interface. The requester then reads this WSDL to understand the available operations, required message formats, data types, and the service endpoint URL.

Interaction with SOAP:

Once the requester has obtained the WSDL from UDDI, it uses that WSDL to construct a SOAP client interface. The WSDL tells the requester how to structure SOAP messages — which operations to call, what parameters to include, and which endpoint URL to send them to. The requester then communicates with the provider via SOAP messages transported over HTTP.

Interaction Flow:



Q8. List and describe the four structural elements of a SOAP message, indicating which are optional and which are required. *(8 marks)*

Answer:

A SOAP message is a standard XML document. Its structure is as follows:

```xml
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

    <soap:Header>
        <!-- Optional: authentication tokens, transaction IDs, routing info -->
    </soap:Header>

    <soap:Body>
        <!-- Required: actual request or response payload -->

        <soap:Fault>
            <!-- Optional: error/exception information -->
        </soap:Fault>
    </soap:Body>
```

```
</soap:Envelope>
```

1. Envelope (Required)

The `<soap:Envelope>` is the root element of every SOAP message. It is mandatory and serves to identify the XML document as a SOAP message. It encapsulates all other elements of the message and declares the necessary XML namespaces (e.g., the SOAP encoding namespace). Without the Envelope, the document is not recognised as a SOAP message.

2. Header (Optional)

The `<soap:Header>` element appears immediately inside the Envelope and before the Body. It carries **metadata** — information about the message itself rather than the payload. Common uses include:

- Authentication and security credentials (WS-Security tokens)
- Transaction identifiers
- Message routing and addressing information
- Priority or retry directives

The Header is processed by SOAP intermediaries or the ultimate receiver as instructed by the sender.

3. Body (Required)

The `<soap:Body>` is mandatory and contains the **actual payload** of the SOAP message — the data being transmitted. In a request message, the Body contains the operation name and its parameters. In a response message, it contains the return values. The Body is the core functional content of the message that the ultimate receiver processes.

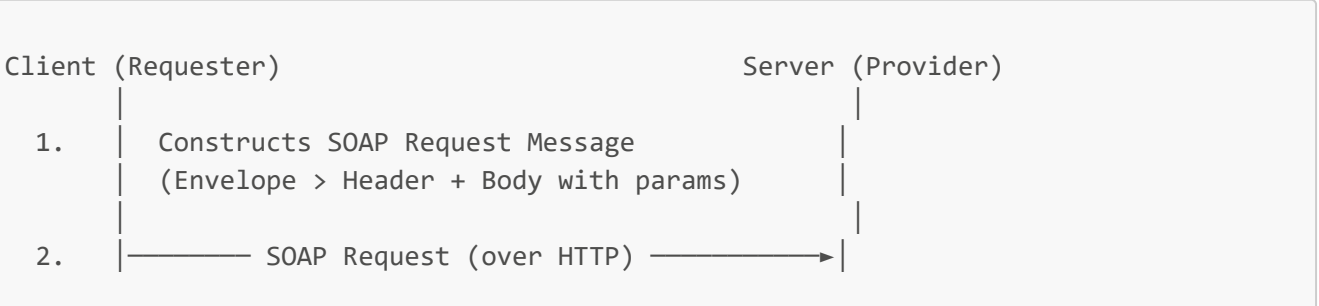
4. Fault (Optional, inside Body)

The `<soap:Fault>` element is an optional child of the Body. It is only present in **response messages** when an error or exception has occurred during processing. It provides structured error reporting, including a fault code, fault string (human-readable description), fault actor (which node generated the fault), and optional detail elements. The standardised Fault structure allows clients to programmatically handle errors.

*(Additionally, SOAP messages can carry **Attachments** in MIME encoding, used for transmitting binary data such as images or files alongside the message.)*

Q9. Describe the six-step SOAP exchange mechanism between a client and a server. *(6 marks)*

Answer:



```
        |                                                    |
        |                                          3.        |  Receives & validates
request
        |                                                    |  Executes business logic
        |                                                    |  (DB query, computation,
etc.)
        |                                                    |
  4.    |◄──────── SOAP Response (over HTTP) ────────────────|
        |                                                    |  Constructs SOAP Response
        |                                                    |  (Envelope > Header +
Body
        |                                                    |   with return values)
  5.    |  Response contains Envelope with payload           |
        |                                                    |
  6.    |  Processes SOAP Response                           |
        |  (extracts return values from Body)                |
```

Step 1: The client constructs a SOAP request message. This message is an XML document structured as a SOAP Envelope containing a Header (optional, for metadata) and a Body (containing the operation name and any input parameters).

Step 2: The client sends the SOAP request message to the server over a network using a transport protocol — most commonly HTTP (POST), but also HTTPS, SMTP, or FTP.

Step 3: The server receives the SOAP request message. It validates the message structure, then processes it — which may involve querying a database, applying business logic rules, or delegating to other services.

Step 4: The server constructs a SOAP response message and sends it back to the client using the same transport protocol used for the request.

Step 5: The SOAP response message contains a SOAP Envelope that defines the structure and the payload — the data being returned (return values, confirmation messages, etc.). If an error occurred, a `<Fault>` element is included in the Body instead.

Step 6: The client receives and processes the SOAP response message, extracting the return values from the Body element for use in its own application logic.

Q10. Describe the seven key elements of a WSDL document, explaining the purpose of each. *(14 marks)*

Answer:

A WSDL (Web Services Description Language) document is an XML-based formal contract that describes a web service. It is logically divided into two parts: **abstract definitions** (what the service does) and **concrete bindings** (how and where to access it).

WSDL Document Structure Diagram:

```
<definitions>  ← Root element (declares namespaces)
|
```

```
├── <types>         ← Abstract: Data type definitions (XSD)
│
├── <message>       ← Abstract: Data units (parameters/return values)
│
├── <portType>      ← Abstract: Logical grouping of operations
│   └── <operation>
│       ├── <input>
│       └── <output>
│
├── <binding>       ← Concrete: Protocol + message format binding
│
└── <service>       ← Concrete: Groups endpoints
    └── <port>      ← Concrete: Physical URL of service
```

---

1. `<definitions>` **(Root Element)**

This is the outermost, root element that encapsulates the entire WSDL document. It declares all the XML namespace prefixes used throughout the document (e.g., the SOAP namespace, the XSD namespace, the target namespace of the service). Every WSDL document must have exactly one `<definitions>` element. In WSDL 2.0, this was renamed to `<description>`.

---

2. `<types>`

This element acts as a container for all data type definitions used by the web service. It primarily uses **XML Schema (XSD)** to define the complex data structures that will be exchanged between the client and the server. For example, if a service operates on an `Order` object containing fields like `orderId`, `item`, and `quantity`, those types are formally defined here. For simple primitive types (String, int), this element may be empty or omitted.

---

3. `<message>`

This element defines the data units being communicated — analogous to the parameters and return values of a function. A `<message>` element consists of one or more `<part>` elements, each representing either a single parameter or the return value. For each operation, there is typically one `<message>` for the input (request) and one for the output (response).

Example:

```xml
<message name="getTimeAsStringRequest"/>
<message name="getTimeAsStringResponse">
    <part name="return" type="xsd:string"/>
</message>
```

---

4. `<portType>` **/** `<interface>` **(Most Critical Element)**

This is considered the most critical element of the WSDL document. It is a logical grouping of the **abstract operations** (methods) provided by the web service. Each `<operation>` inside the `<portType>` references the input and output messages defined earlier. The `<portType>` defines *what* the service can do, independently of *how* or *where* it does it. In WSDL 2.0, this element was renamed to `<interface>`, and it additionally supports inheritance.

Example:

```
<portType name="TimeServerInterface">
    <operation name="getTimeAsString">
        <input message="tns:getTimeAsStringRequest"/>
        <output message="tns:getTimeAsStringResponse"/>
    </operation>
</portType>
```

---

5. `<binding>`

This element bridges the gap between the abstract `<portType>` and the concrete protocol used to transmit messages. It specifies:

- **Which protocol** is used (e.g., SOAP over HTTP)
- **What message format** is used (e.g., RPC style or Document style)
- **SOAP-specific details** for each operation (e.g., `soapAction`)

The `<binding>` element links an abstract operation definition to a concrete transport and encoding mechanism.

---

6. `<service>`

This element groups one or more related **endpoints (ports)** together under a named service. A `<service>` element represents the complete, deployable web service, and it may contain multiple `<port>` elements if the same service is accessible at different endpoints or via different bindings.

---

7. `<port>` **/** `<endpoint>`

This element defines a single, concrete communication endpoint. It associates a physical **network address (URL)** with a specific `<binding>`. This is where a client will actually send requests. In WSDL 2.0, `<port>` was renamed to `<endpoint>`.

Example:

```
<service name="TimeServerService">
    <port name="TimeServerPort" binding="tns:TimeServerBinding">
        <soap:address location="http://localhost:9873/ts"/>
    </port>
</service>
```

WSDL Abstract vs Concrete Division:

WSDL Element	Division	Answers
`<types>`	Abstract	What data types are exchanged?
`<message>`	Abstract	What data units are transmitted?
`<portType>`	Abstract	What operations are available?
`<binding>`	Concrete	How are messages transmitted (protocol/format)?
`<service>` + `<port>`	Concrete	Where is the service located (URL)?

Q11. Distinguish between the Top-Down (Contract-First) and Bottom-Up (Code-First) approaches to WSDL development, including an example tool for each. *(4 marks)*

Answer:

Top-Down (Contract-First):

In this approach, the developer begins by designing and writing the **WSDL file first**, before any implementation code exists. The WSDL represents the agreed-upon service contract — it defines the operations, data types, and communication protocol. Once the WSDL is finalised, tools are used to automatically generate the Java code scaffolding (stubs and skeletons) from it.

This approach is preferred in **enterprise and team environments** where the service interface must be agreed upon between providers and consumers before development begins. It guarantees that the generated code is fully compliant with the contract.

- **Example tool**: `wsimport` (JDK) or `WSDL2Java` (Apache Axis)
- Workflow: `WSDL file → wsimport → Java SEI + Service stubs`

Bottom-Up (Code-First):

In this approach, the developer begins by writing the **Java implementation code first** — typically a class with business logic — and then uses tools to automatically generate the WSDL from the annotated code.

This approach is simpler and faster for individual developers or when converting an existing Java class into a web service. The risk is that the resulting WSDL may be less clean or interoperable than one designed contract-first.

- **Example tool**: `wsgen` (JDK) or `Java2WSDL` (Apache Axis)
- Workflow: `Java class (@WebService) → wsgen → WSDL + JAXB classes`

Q12. List the four WSDL operational patterns and briefly describe the message flow for each. *(8 marks)*

Answer:

WSDL defines four **transmission primitives** that describe how messages flow between the client and the service:

1. One-Way

The client sends a message to the service, and the service processes it. **No response is returned.** This is a fire-and-forget pattern. Suitable for logging events, notifications, or triggering background processing where the client does not need confirmation.

```
Client ──────► Service
        (message)
                ← (no response)
```

2. Request-Response (Most Common)

The client sends a request message to the service, and the service processes it and **returns a response message**. This is the standard, most widely used pattern in web services — it mirrors a traditional function call. All examples in this course (TimeServer, HelloWorld, Calculator, OrderService) use this pattern.

```
Client ──────► Service
       (request)
Client ◄────── Service
       (response)
```

3. Solicit-Response

In this pattern, it is the **service** (provider) that initiates communication by sending a request message to the client. The client then processes the request and returns a response. This is the reverse of the normal Request-Response pattern. It is less common and used when the service needs to push information to clients or request an action from them.

```
Service ──────► Client
        (solicit)
Service ◄────── Client
        (response)
```

4. Notification

The service sends a message to the client (or multiple clients) **without expecting any response**. This is a publish-subscribe or broadcast pattern, used for event notifications, alerts, or broadcasting state changes to interested parties.

```
Service ──────► Client
        (notification)
                ← (no response expected)
```

Q13. Name and distinguish the two main Java APIs for web services development, specifying the web service style each targets and their primary implementations. *(6 marks)*

Answer:

Java defines two main APIs for web services development under the Java EE/Jakarta EE specification:

1. JAX-WS (Java API for XML Web Services)

JAX-WS is the Java standard API for building **SOAP-based web services**. It is commonly abbreviated as JWS (Java Web Services). All libraries required to compile, publish, and consume JAX-WS services are available in core Java SE 6 and above, making it accessible without additional dependencies for basic use.

JAX-WS supports two styles of SOAP binding:

- **RPC Style** (`SOAPBinding.Style.RPC`): maps operations to remote method calls; supports simple data types only.
- **Document Style** (`SOAPBinding.Style.DOCUMENT`): sends full XML documents; supports complex types; the industry default.

Key annotations: `@WebService`, `@WebMethod`, `@SOAPBinding`.

2. JAX-RS (Java API for RESTful Web Services)

JAX-RS is the Java standard API for building **RESTful web services**. It is designed around HTTP methods (GET, POST, PUT, DELETE) and resource-based URLs, using JSON or XML for data exchange. JAX-RS is not included in core Java SE and requires a separate implementation.

The two main implementations are:

- **Jersey** (the official JAX-RS reference implementation, developed by Oracle/Eclipse)
- **RESTeasy** (developed by Red Hat / JBoss)

Modern Context:

While JAX-WS and JAX-RS are the official Java EE standards, most modern Java developers use **Spring Boot** to build web APIs, as it provides a more productive and convention-based development experience. Spring Boot is not covered in this course.

Comparison:

Feature	JAX-WS	JAX-RS
Service style	SOAP-based	RESTful
Message format	XML (SOAP Envelope)	JSON, XML
Transport	HTTP, SMTP, JMS, TCP	HTTP/HTTPS only
Included in Java SE	Yes (Java 6+)	No (needs Jersey/RESTeasy)

Feature	JAX-WS	JAX-RS
Key implementations	RPC Style, Document Style	Jersey, RESTeasy

---

Part II: Essay Questions — Full Model Answers

Essay 1. Critically compare SOAP and REST as approaches to building web services, covering data formats, transport protocols, state management, security models, performance, and error handling. Conclude with a justified recommendation for: (a) a mobile banking application, (b) a public social media API. *(25 marks)*

Introduction

SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) represent the two dominant paradigms for building web services. SOAP is a **strict, standards-based protocol** governed by W3C specifications, while REST is a **flexible architectural style** built on the conventions of the HTTP protocol. The choice between them is not merely technical but strategic — shaped by security requirements, data complexity, performance constraints, and the organisational context of the application.

---

1. Data Formats

SOAP mandates the use of **XML exclusively** as its message format. Every SOAP message is wrapped in a verbose XML Envelope structure containing a Header, Body, and optionally a Fault element. This structure guarantees a well-defined, standardised message format, but it comes at the cost of verbosity. Even a simple request to retrieve a single value requires a full Envelope structure.

REST, by contrast, is **format-agnostic**. It most commonly uses **JSON** (JavaScript Object Notation), which is significantly more compact and human-readable than XML. REST can also use XML, HTML, plain text, or binary formats depending on the use case. JSON has become the dominant choice in modern APIs due to its lightweight nature and native compatibility with JavaScript in browser and mobile environments.

Illustration — Retrieving User 123:

REST Request:

```
GET /users/123 HTTP/1.1
Host: api.example.com
```

REST Response:

```
{"id": 123, "name": "Jane Doe", "email": "jane@example.com"}
```

SOAP Request:

```
POST /UserService HTTP/1.1
<soap:Envelope xmlns:soap="...">
    <soap:Body>
        <GetUserRequest><UserId>123</UserId></GetUserRequest>
    </soap:Body>
</soap:Envelope>
```

The contrast in verbosity is stark and has direct implications for bandwidth consumption and processing overhead.

---

2. Transport Protocols

SOAP is **transport-neutral**. While HTTP is the most common transport, SOAP messages can be sent over SMTP (email), JMS (Java Message Service), FTP, TCP, or any other protocol. This makes SOAP suitable for complex enterprise integration scenarios where multiple transport channels may be required.

REST is **exclusively tied to HTTP/HTTPS**. Every REST interaction uses standard HTTP methods: GET (retrieve), POST (create), PUT (update), DELETE (delete). While this constrains REST to HTTP, it also means REST can leverage all HTTP infrastructure — load balancers, proxies, CDNs, and browsers — without any special configuration.

---

3. State Management

SOAP can be **stateful or stateless** depending on the implementation. SOAP services can maintain session state between requests using WS-Addressing or custom Header elements, which is important for multi-step transactions such as booking systems or financial workflows.

REST is **inherently stateless**. Every REST request must contain all the information needed for the server to process it — no client session state is stored on the server. This statelessness is a fundamental REST constraint and is one of the reasons REST scales so well horizontally: any server in a cluster can handle any request independently.

---

4. Security Models

SOAP provides **WS-Security**, a W3C specification for message-level security. WS-Security supports XML Digital Signatures (message integrity), XML Encryption (message confidentiality), and security tokens (authentication). Crucially, this security is applied at the **message level**, meaning messages remain secure even when passing through intermediaries or multiple transport hops. SOAP also natively supports **ACID (Atomicity, Consistency, Isolation, Durability)** transactional properties through WS-AtomicTransaction.

REST relies primarily on **transport-level security** via HTTPS (SSL/TLS). Modern REST APIs also use OAuth 2.0 for delegated authorisation and JWT (JSON Web Tokens) for stateless authentication. While HTTPS is widely sufficient for most use cases, REST security does not protect messages once they leave the encrypted tunnel — a limitation in environments with complex intermediary chains.

---

5. Performance

REST generally outperforms SOAP in terms of speed and resource consumption for several reasons:

- **JSON payloads are smaller** than equivalent SOAP/XML messages, reducing bandwidth usage.
- REST responses can be **cached** using standard HTTP caching mechanisms (ETags, Cache-Control headers), dramatically reducing server load for frequently accessed resources.
- REST has **less processing overhead** — no XML parsing of envelope structures, no WSDL contract negotiation.

SOAP's XML envelopes impose constant parsing overhead, and SOAP responses are explicitly marked as non-cacheable by default.

---

6. Error Handling

SOAP provides a **standardised `<Fault>` element** within the message Body for error reporting. A Fault contains a structured set of fields: `faultcode` (machine-readable error classification), `faultstring` (human-readable description), `faultactor` (which node generated the error), and `detail` (additional information). This standardisation means all SOAP consumers can handle errors in a consistent, predictable way.

REST relies on **standard HTTP status codes** (e.g., 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error) supplemented by custom JSON error response bodies. While HTTP status codes are widely understood, the custom error body format varies between APIs, meaning error handling is less standardised across different REST services.

Comparison Summary Table:

Feature	SOAP	REST
Data Format	XML only (verbose)	JSON, XML, HTML, Text (flexible)
Transport	HTTP, SMTP, JMS, FTP, TCP	HTTP/HTTPS only
State	Stateful or stateless	Inherently stateless
Security	WS-Security (message-level)	HTTPS, OAuth 2.0, JWT (transport-level)
Performance	Slower (heavy XML, no caching)	Faster (lightweight JSON, HTTP caching)
Contract	Required (WSDL)	Optional (OpenAPI/Swagger)
Error Handling	Standardised `<Fault>` element	HTTP status codes + custom bodies
Caching	Not cacheable	HTTP caching supported

Scenario Recommendations:

(a) Mobile Banking Application → SOAP is the appropriate choice.

Banking transactions are mission-critical operations requiring:

- **ACID compliance** — financial transactions must be atomic (all-or-nothing), consistent, isolated, and durable. SOAP's WS-AtomicTransaction specification directly supports this.

- **End-to-end message-level security** — with SOAP and WS-Security, messages are encrypted and signed at the message level, not just the transport level. This protects sensitive data even if it passes through multiple intermediaries in a bank's internal network.
- **Strict formal contracts** — WSDL contracts ensure that all parties (client apps, core banking systems, regulatory systems) communicate with an agreed-upon, versioned, and validated interface. Any breaking change is immediately detectable.
- **Compliance** — many banking and financial regulatory frameworks (PCI-DSS, ISO 20022) align with SOAP-based web service standards.

(b) Public Social Media API → REST is the appropriate choice.

A public social media API must serve millions of requests from web browsers, mobile apps, and third-party developers. REST is optimal because:

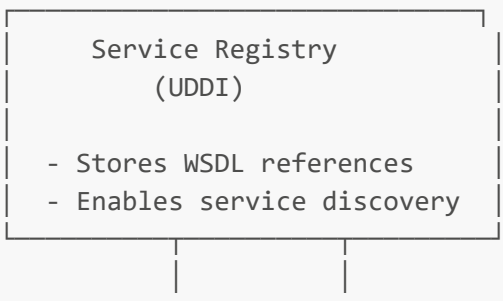
- **Performance at scale** — JSON payloads are compact and HTTP caching enables CDN-level caching of popular resources (e.g., trending posts), dramatically reducing server load.
- **Ease of consumption** — REST over JSON with OAuth 2.0 is immediately consumable from any platform — web browsers (JavaScript), mobile apps (Swift, Kotlin), and scripting environments — without SOAP libraries.
- **Statelessness** — RESTful statelessness allows horizontal scaling across thousands of servers with no session affinity requirements.
- **Developer experience** — public APIs must be easy for third-party developers to adopt. REST's resource-based URL design (`GET /posts/123`, `POST /comments`) is intuitive and well-documented with tools like OpenAPI/Swagger.

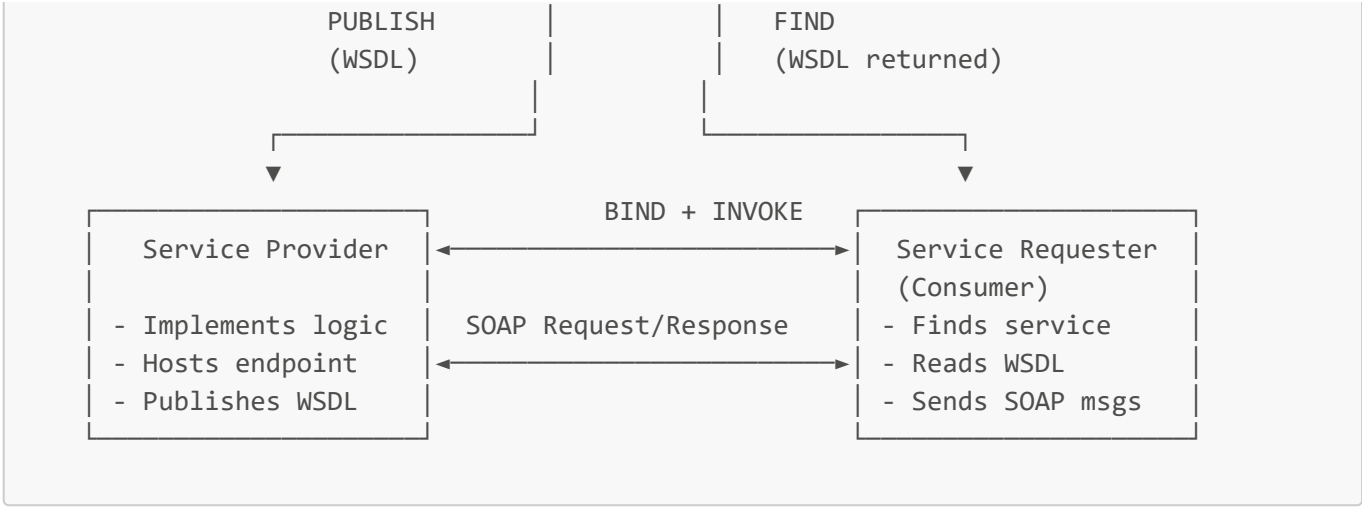
Essay 2. Describe the architecture of web services as realised through SOA, explaining the SOA roles, the role of SOAP/WSDL/UDDI as enabling technologies, and the practical steps involved in implementing a web service from design to consumption. *(25 marks)*

Introduction

Service-Oriented Architecture (SOA) is an architectural style that designs software systems as a collection of **independent, reusable, interoperable services** aligned with business functionality rather than technical implementation. Web services are the primary technology used to implement and realise SOA in practice. The SOA model defines clear roles, interactions, and standards that together enable loosely coupled, platform-independent distributed computing.

The Three SOA Roles and Their Interactions





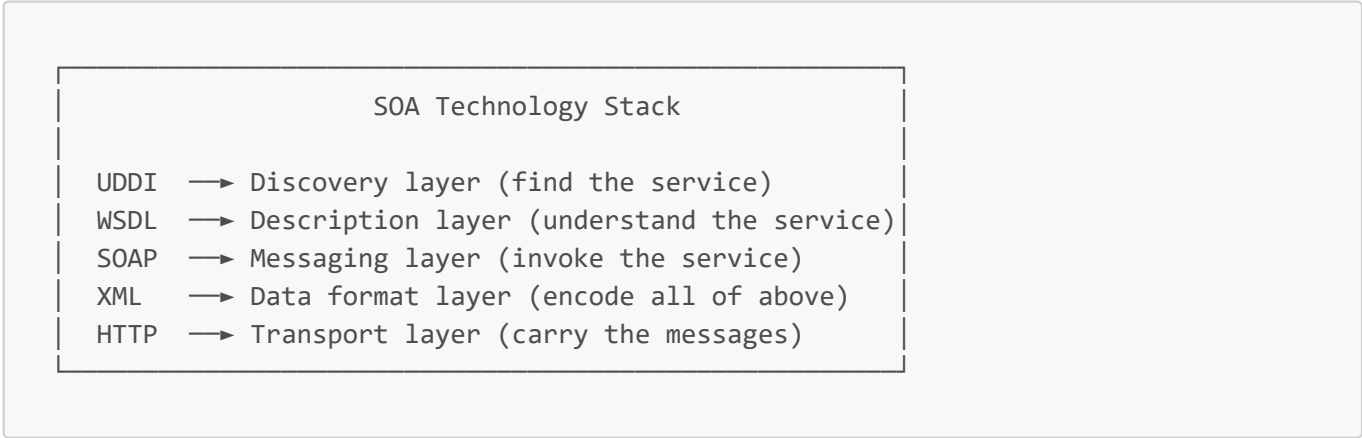
Service Provider: The Provider is responsible for developing, deploying, and hosting the web service. It implements the business logic (e.g., a payment processing function, a flight availability check), exposes it as a web service endpoint, and publishes a WSDL document to the UDDI registry so potential consumers can discover and understand the service.

Service Broker / Registry (UDDI): The Broker acts as the directory that connects providers with consumers. When providers register, they submit service metadata and WSDL pointers. When consumers search, they receive WSDL descriptions back. UDDI uses an XML-based data model and is the standardised mechanism for web service registration and discovery.

Service Requester / Consumer: The Consumer application discovers services via the UDDI registry, retrieves their WSDL descriptions, and uses those descriptions to bind to the provider's endpoint. It then invokes the service by sending SOAP messages and processes the responses.

The Enabling Technologies: SOAP, WSDL, UDDI as a Stack

These three standards work together as a complementary stack:



SOAP (Simple Object Access Protocol) is the messaging protocol. It defines a standardised XML envelope structure for packaging request and response data. SOAP provides the actual communication channel — it carries the operation invocations and their results. It is transport-neutral (HTTP, SMTP, JMS) and platform-independent.

WSDL (Web Services Description Language) is the description and contract language. A WSDL document is a machine-readable XML file that formally describes a service's available operations, the message formats for

each operation, the data types involved, and the physical endpoint URL. WSDL is how a consumer learns what a service can do and how to communicate with it. It is the bridge between UDDI discovery and SOAP invocation.

UDDI (Universal Description, Discovery, and Integration) is the discovery mechanism. It provides a standardised registry where providers list their services (with WSDL references) and consumers search for services. Querying UDDI returns WSDL descriptions, which then enable the consumer to build a SOAP client.

Together, the workflow is: **UDDI enables finding → WSDL enables understanding → SOAP enables invoking.**

Practical Implementation Lifecycle: Design to Consumption

Step 1 — Requirements and Design

The developer identifies the business functionality to expose (e.g., a customer lookup service, a payment gateway). Key decisions include:

- **Architecture choice**: SOAP-based (for enterprise, security-critical systems) or RESTful (for modern, lightweight APIs).
- **Contract design**: For SOAP, this means designing or generating the WSDL. Choosing Top-Down (WSDL first) ensures a clean, agreed-upon interface before coding begins.

Step 2 — Development (Coding)

- Write the **Service Endpoint Interface (SEI)**: a Java interface annotated with `@WebService`, `@SOAPBinding`, and `@WebMethod` for each operation.
- Write the **Service Implementation Bean (SIB)**: a Java class implementing the SEI, annotated with `@WebService(endpointInterface="...")`, containing the actual business logic.
- Handle data type mapping: ensure complex objects are properly bound to XML schema types via JAXB.

Step 3 — Deployment

- For development: use Java SE's built-in HTTP server via `Endpoint.publish(url, new SIBInstance())`.
- For production: deploy to an application server (GlassFish, JBoss, WebSphere, Tomcat) which provides lifecycle management, connection pooling, EJB container services, and security enforcement.

Step 4 — Testing and Validation

- Access the auto-generated WSDL at `http://host:port/path?wsdl` to verify the contract.
- Use tools such as **SoapUI** or **Postman** to send test requests and verify responses.
- Validate security (authentication, authorisation, SSL) and exception handling (Fault responses).

Step 5 — Discovery and Consumption

- Optionally publish the service to a **UDDI registry** for broader discoverability.
- Client developers receive the WSDL URL and run `wsimport` to generate client stubs.
- Generated stubs allow calling the remote service as if it were a local Java method.

Critical Reflection

In modern enterprise practice, pure UDDI registries have largely been replaced by internal service catalogues, API gateways (e.g., AWS API Gateway, Kong), and REST/OpenAPI ecosystems. The SOAP/WSDL/UDDI stack remains vital in legacy system integration, particularly in banking, healthcare, and government systems built in the early 2000s. Understanding this stack is essential for maintaining, extending, or migrating such systems.

Essay 3. Web services are described as being "language-agnostic, vendor-neutral, and transport-neutral." To what extent do these properties hold true in practice? *(20 marks)*

Introduction

The defining promises of web services — language-agnosticism, vendor neutrality, and transport neutrality — are foundational to their appeal as integration technology. At the standards and protocol level, these properties are largely well-realised. However, practical implementation experience reveals a more nuanced picture, where architectural choices, tooling constraints, and deployment realities can partially compromise these ideals.

Language-Agnosticism

In theory, a SOAP web service built in Java should be consumable by a .NET client, a Python script, a PHP application, or a Perl program, as illustrated in the lecture examples (the TimeServer was consumed by both a Java client and a Perl client using `SOAP::Lite`). The XML-based SOAP messaging format is parseable by any language with an XML library, and WSDL is also language-independent.

In practice, this property holds strongly at the protocol level. However, minor discrepancies can emerge in:

- **Data type mapping**: Different languages map XML Schema types to native types differently. For example, `xsd:dateTime` may be mapped to `java.util.Date` in Java but to `DateTime` in .NET, with subtle differences in timezone handling.
- **JAXB and marshalling**: The JAXB binding framework in Java handles XML-to-object mapping, but the generated binding classes are Java-specific. A Python client must independently parse the same XML, potentially introducing inconsistencies.

Overall, language-agnosticism is one of the most successfully realised properties of web services, provided developers adhere to WS-I Basic Profile compliance.

Vendor Neutrality

The SOA model centres on open, vendor-neutral standards (W3C SOAP, OASIS UDDI, W3C WSDL). In principle, a service hosted on Oracle WebLogic should be consumable by a client built with Apache CXF, IBM WebSphere, or any other compliant framework.

In practice, vendor neutrality is partially compromised by:

- **Proprietary extensions**: Vendors often add non-standard extensions to their SOAP implementations (e.g., vendor-specific WSDL extensions or security headers). Services that use these extensions become tied to that vendor's ecosystem.

- **WSDL interpretation differences**: While WSDL is a standard, different JAX-WS implementations (Metro, Apache CXF, Axis2) can generate slightly different WSDL from the same annotated Java code, leading to interoperability issues.
- **Application server differences**: Deploying on GlassFish versus JBoss versus WebSphere can require different configuration, deployment descriptors, and annotations, partially defeating the vendor-neutral ideal.

Transport Neutrality

SOAP is theoretically transport-neutral — it can run over HTTP, SMTP, JMS, FTP, and TCP. This is a genuine architectural strength for enterprise integration, where messages may need to travel through email systems (SMTP) or message queues (JMS) as well as HTTP.

However, in REST-based web services, transport neutrality is abandoned entirely — REST is exclusively HTTP/HTTPS. This represents a pragmatic trade-off: by committing to HTTP, REST services gain the full benefit of HTTP infrastructure (caching, content negotiation, standard status codes, CDN support).

Even within SOAP, the vast majority of real-world implementations use HTTP as the transport. The other protocols are theoretically available but rarely used in practice.

Furthermore, the **RPC-based communication model** introduces tight coupling at the communication level — clients must know specific method signatures, which creates a form of interface dependency that partially undermines the transport-neutral ideal.

Conclusion

The three neutrality properties hold most strongly at the **theoretical/standards level** and degrade incrementally as implementation specifics are introduced. Language-agnosticism is the most consistently realised. Vendor neutrality is strong for standard-compliant implementations but weakened by proprietary extensions and tooling differences. Transport neutrality is a real advantage for SOAP in complex enterprise integrations but irrelevant for REST-based services.

The key insight for a practising engineer is that these ideals are **aspirational design principles** that guide architecture toward interoperability. They should be actively preserved through strict adherence to open standards, avoidance of proprietary extensions, and thorough testing across heterogeneous environments. The ideals are worth pursuing precisely because the costs of not doing so — vendor lock-in, integration fragility, and reduced portability — are high in long-lived enterprise systems.

End of Chapter 2 Full Model Answers